I’ve just completed a blueprint on building high-performance, scalable mobile apps using React Native, Expo, and Convex. This isn't just about basic CRUD apps—we're talking about a "Universal Full-Stack" approach that integrates:

🤖 Agentic AI with financial guardrails
⚡ Real-time Reactivity (no more manual polling!)
⚛️ Quantum Computing workflows
🌐 TanStack integration for a unified web/mobile codebase

If you're looking to modernize your stack or are curious about where mobile dev is heading, check out the full analysis below. 👇

#ReactNative #Expo #Convex #TanStack #GraphDatabase #MobileDev #SoftwareArchitecture #AI

The stack is built on React Native and Expo for the frontend, and Convex as the reactive backend and orchestration layer.

Velocity: Rapid development via Expo's managed workflow and Convex's serverless functions.
Reactivity: Real-time data synchronization as a default, eliminating manual polling.
Performance: Strict adherence to 60+ FPS UI by offloading heavy computation (AI, Quantum processing, Crypto) to background threads (Worklets) or backend services.
Agentic AI: A forward-looking architecture designed to support autonomous AI agents with financial capabilities and strict guardrails.

2. Core Architecture

The documentation defines several specialized architectural patterns tailored to specific domains:

2.1 Three-Layer Reactive Agent Architecture (TLRAA)

Designed for integrating AI agents, this architecture separates concerns into:

Presentation Layer (Expo/React Native): Handles UI and user input.
Synchronization/Orchestration Layer (Convex): Acts as the source of truth, managing state, transactions, and coordinating external services.
Agentic Microservice Layer: External, scalable services (e.g., CrewAI, LangGraph) that perform heavy LLM inference and complex reasoning.

2.2 Quantum-Reactive Architecture (QRA)

Addresses the extreme latency of Quantum Processing Units (QPUs) by decoupling the client from the backend:

Client: Remains responsive using optimistic updates and reactive subscriptions.
Backend: Convex manages long-running, asynchronous jobs (queuing, polling, result processing) using durable Workflows.

2.3 Unified Payment Architecture (UPA)

Integrates standard user payments with novel "Agentic" payments:

User Payments: Uses RevenueCat for subscriptions and Stripe for transactions, secured by Convex Actions and Webhooks.
Agentic Payments: Introduces "Financial Guardrails" where autonomous agents must pass atomic budget checks (Mutations) before executing external spending.

3. Technology Stack & RolesArchitecture

Minimize image
Edit image
Delete image

4. Key Integrations

4.1 AI & Large Language Models (LLMs)

Orchestration: Convex Actions serve as the secure gateway for all LLM interactions.
Streaming: Implements "Persistent Text Streaming" to deliver token-by-token responses via Convex's reactive tables.
RAG: Native Vector Search in Convex allows for Retrieval-Augmented Generation without external vector databases.
Type Safety: Enforces end-to-end type safety using Pydantic (Python agents) and Zod (TypeScript backend/client).

4.2 Payments & Financial Security

RevenueCat: Standard for cross-platform subscriptions (iOS/Android/Web).
Webhooks: Convex HTTP Actions securely ingest webhooks from Stripe/RevenueCat to trigger fulfillment Workflows.
Agentic Guardrails:Atomic Check-and-Debit: A Mutation that transactionally verifies and deducts budget before any external API call.Audit Ledger: Immutable log of all agent-initiated transactions for non-repudiation.

4.3 Quantum Computing

Asynchronous Workflows: Handles QPU queue times (seconds to hours) using Convex Scheduled Actions and polling loops.
Hybrid Jobs: Supports submission of hybrid quantum-classical jobs for priority execution.
Visualization: Client uses Worklets to process raw quantum measurement data (e.g., Qiskit Counts) into probability distributions for smooth UI rendering.

4.4 Graph Databases (GDB)

Hybrid Data Model: Convex handles high-velocity mutable state; GDB handles complex relationship queries.
CQRS Pattern: Writes go to Convex (Mutations); complex reads are proxied via Convex Actions to the GDB.
CDC Pipeline: Changes in the GDB are streamed back to Convex to update reactive views.

4.5 Device Utilities

Native Access: Uses Expo Config Plugins to manage permissions (Camera, GPS, Sensors).
Performance: High-frequency sensor data is processed via Worklets to avoid blocking the JS thread.
File Uploads: Implements a secure 3-step upload protocol for large binary files.

5. Performance & User Experience (UX)

Thread Integrity: The "Golden Rule" is to never block the JS thread. Heavy tasks (crypto, parsing, physics) must use Worklets.
React Compiler: Enabled by default (Expo SDK 54+) to optimize rendering and minimize reconciliation costs.
Optimistic UI: Leveraging Convex's instant reactivity to provide immediate feedback before server confirmation.
Native Fidelity: Utilizing Expo Router for native navigation patterns (e.g., large collapsing headers) and Reanimated for shared element transitions.

6. Security & Compliance

Row-Level Security (RLS): Enforced in Convex Queries/Mutations using ctx.auth.getUserId() and withIndex.
Secret Management: API keys (Stripe, OpenAI, Quantum Providers) are stored in Convex Environment Variables, never on the client.
Compliance:AML/KYC: Automated agentic checks integrated into payment workflows.Audit Trails: Comprehensive logging of all financial and agentic actions.

7. TanStack Integration (Universal Architecture)

The "Universal Full-Stack Monorepo" pattern integrates TanStack Start (TSS) with Expo and Convex to create a unified, type-safe development environment for Web and Mobile.

7.1 Core Roles

TanStack Start (TSS): Serves as the unified framework. On Web, it handles SSR. For Mobile, it acts as a deployed, type-safe RPC/API layer (via Server Functions) for operations outside Convex's scope.
Expo: Manages the Native runtime and build process (EAS).
Convex: Remains the primary "Sync Engine" for real-time data and transactional state.

7.2 Integration Strategy

Routing Reconciliation:Native: Uses Expo Router (built on React Navigation).Web: Uses TanStack Router.Strategy: A shared app/ directory with a custom adapter to force TanStack Router to respect Expo's file-based routing conventions (e.g., _layout.tsx, (group)).
Data Fetching:Convex Adapter for TanStack Query: Replaces standard polling with Convex's push-based reactivity.Configuration: isStale: false (data is never stale), gcTime tuned to maintain subscription continuity during navigation.Hooks: useSuspenseQuery for critical initial render data; useQuery for lazy updates.

7.3 RPC Decision Hierarchy

To prevent overlap between Convex Actions and TSS Server Functions, the architecture mandates:

Convex Mutation: MUST be used for all transactional DB writes.
Convex Action: MUST be used for external side effects (e.g., Stripe calls) that are part of a business workflow.
TSS Server Function: ONLY used for:Proxying non-database 3rd party APIs.Accessing backend secrets not available to Convex.Complex external auth flows (e.g., WorkOS) unsuitable for Convex Actions.

7.4 Performance Enhancements

Remote RPC: Mobile clients use VITE_SERVER_BASE_URL to call TSS Server Functions.
Reactive Jitter Mitigation: Worklets are used to process/normalize high-frequency reactive data streams from Convex before they trigger a React render.